Oggetti e Classi

Introduzione

Abbiamo lavorato sugli array e visto la sintassi di base, oggi vedremo le classi e oggetti.

Approcci (Paradigmi di programmazione)

Un paradigma di programmazione è uno stile di programmazione.
È costituito da:

- **un insieme di concetti, principi e regole utilizzati prima dal progettista e poi dal programmatore per la realizzazione di un software.**
  Il paradigma:
- **definisce come si modella un problema**
- **influenza come si concepisce la soluzione**
- **guida come si realizza il programma**

I principali paradigmi della programmazione sono:

- **procedurale (o funzionale)**
- **object-oriented (OOP)**

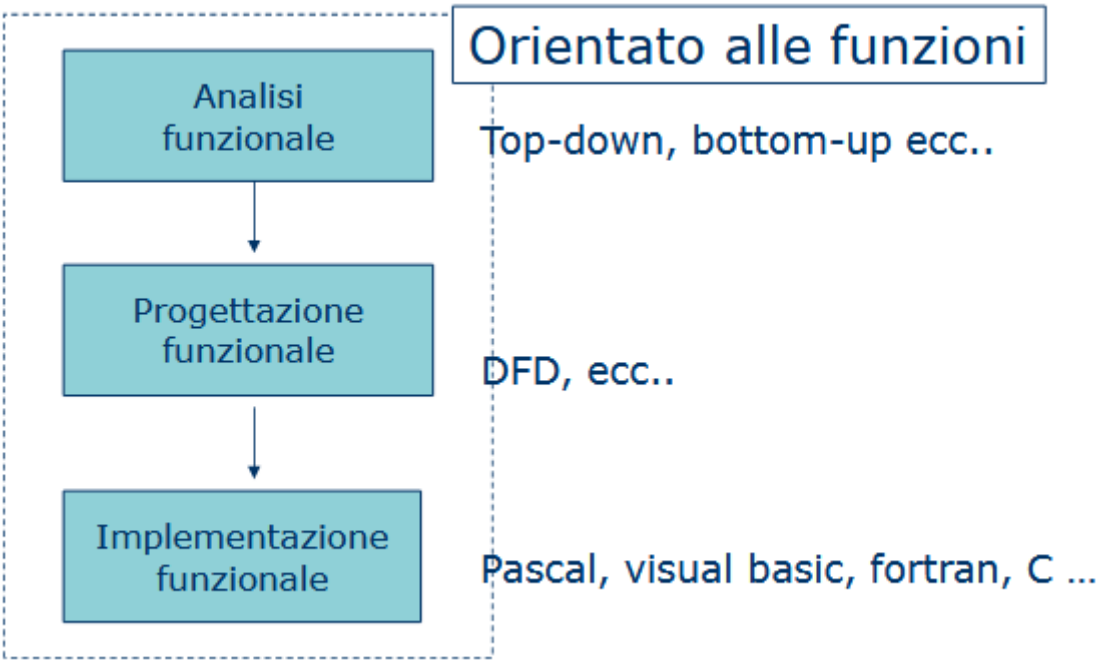
Ruolo dell'approccio nel ciclo di vita del software

L'approccio scelto **condiziona tutta la parte centrale del ciclo di vita del software**, dall'analisi dei requisiti fino all'implementazione.

Per questo motivo:

- **il paradigma non è solo un modo di scrivere codice**

- **ma è soprattutto un modo di progettare il software**



Perché esistono principalmente due approcci?

La programmazione può essere ricondotta a due elementi fondamentali:

1. **dati**
2. **funzioni**

Questi due aspetti sono **principali e complementari**.

Paradigma procedurale / funzionale

- **Le funzioni sono i protagonisti**
- **I dati vengono passati come parametri alle funzioni**
- **Il programma è visto come una sequenza di operazioni**

Paradigma Object-Oriented (OOP)

- **I dati sono i protagonisti**
- **Le funzioni (metodi) sono subordinate ai dati**
- **Le funzioni non possono operare se non esiste un oggetto**
- **Il comportamento è sempre associato a un'entità**

Per questo motivo **è improbabile la nascita di un "terzo" paradigma** completamente distinto:

- **tutti i modelli di programmazione ruotano attorno a dati e funzioni, dando priorità all'uno o all'altro.**

Considerazioni storiche

Storicamente, il **paradigma procedurale** è stato il primo a nascere, perché:

- **è più vicino al funzionamento della macchina**
- **riflette direttamente il linguaggio macchina**
- **richiede un livello di astrazione minore**

Anche il paradigma funzionale, pur introducendo concetti più avanzati, rimane a un livello relativamente basso e richiede una **maggiore capacità di astrazione** da parte del programmatore.

Il paradigma **orientato agli oggetti**, invece, risulta spesso più intuitivo, perché:

- **il mondo reale è popolato da entità (Persona, Animale, Veicolo, ecc.)**
- **il software viene modellato come un insieme di oggetti che collaborano**

In conclusione:

Un paradigma di programmazione:

- **non è solo un modo di pensare**
- **ma anche un modo di agire**
- **fornisce linee guida precise su come analizzare, progettare e implementare un software**

La scelta del paradigma influisce profondamente sulla struttura del codice e sulla sua evoluzione nel tempo.

Esempi di approccio funzionale e orientato agli oggetti

Per chiarire la differenza tra **scomposizione funzionale** e **approccio orientato agli oggetti**, consideriamo come esempio un applicativo complesso come **Amazon**.

Approccio funzionale (scomposizione funzionale)

Nell'approccio funzionale **il problema viene analizzato partendo dalle funzioni che il sistema deve svolgere**.

Le domande principali da porsi sono:

- Quali sono le **funzioni principali** del sistema?
- Quali sono le **sotto-funzioni** che compongono ciascuna funzione?

Il software viene quindi progettato come:

- **un insieme di funzioni**
- **organizzate gerarchicamente**
- **che operano su dati passati come parametri**

> 🎟 **L'attenzione è rivolta al flusso delle operazioni.**

Approccio orientato agli oggetti (OOP)

L'approccio orientato agli oggetti **non parte dalle funzioni**, **ma dalle entità del dominio**.

Il processo di analisi prevede:

- **individuare i protagonisti del problema**
- **definire come sono fatti** → *caratteristiche (attributi)*
- **definire cosa fanno** → *comportamenti (metodi)*

Riprendendo l'esempio di Amazon, l'analisi OOP si articola in:

- Quali sono i **protagonisti principali** del sistema?
- Quali **caratteristiche** li descrivono?
- Quali **azioni** compiono?

Le ultime due domande rappresentano le **specifiche** della prima, ovvero servono a descrivere in modo completo ogni protagonista individuato.

Confronto concettuale

- Nell'approccio funzionale:
  - **le funzioni sono centrali**
- **i dati sono secondari e vengono passati alle funzioni**
- Nell'approccio OOP:
  - **i dati (oggetti) sono centrali**
  - **le funzioni esistono solo come comportamenti degli oggetti**

Classificazione

L'**analisi orientata agli oggetti** passa attraverso il processo di **classificazione**.

Durante questa fase:

- **tutti gli oggetti che condividono le stesse caratteristiche e le stesse funzionalità**
- **vengono raggruppati in una stessa categoria**

La classe

Una **classe** è il risultato del processo di classificazione e:

- **rappresenta una categoria di oggetti concreti appartenenti al dominio applicativo**
- **descrive ciò che gli oggetti hanno in comune**
- **non rappresenta un singolo oggetto, ma un modello**

La classe è quindi un'**astrazione** di oggetti concreti del dominio.

Esempio

Nel contesto di un applicativo come Amazon:

- **i singoli venditori reali sono entità concrete del dominio**
- la classe `Venditore` rappresenta l'**astrazione** comune a tutti questi soggetti

La classe raccoglie:

- **le caratteristiche condivise (attributi)**
- **le funzionalità comuni (metodi)**

In conclusione

Il processo di classificazione consente di:

- **semplificare la complessità del dominio**
- **individuare le entità rilevanti**
- **costruire un modello software coerente e riutilizzabile**

Classe e oggetto

Quando si descrive un oggetto ci si riferisce **sempre alla sua classificazione**. Per poter utilizzare un oggetto, è necessario fare riferimento a un **rappresentante concreto della categoria**.

Esempio:

- Non si dice:
  "Questa penna scrive"
- Si dice:
  "Le penne servono per scrivere"

Se voglio realmente scrivere, devo procurarmi **una penna concreta**, cioè un **oggetto**, e non la categoria astratta delle penne.

Definizioni

- **Classe**: **rappresenta la categoria astratta di appartenenza degli oggetti; descrive caratteristiche e comportamenti comuni.**
- **Oggetto**: **è l'entità concreta, un rappresentante della sua classe; possiede valori reali degli attributi e può eseguire i comportamenti definiti nella classe.**

In altre parole:

> **L'oggetto è un'istanza concreta della sua classe di riferimento.**

Costruzione delgi oggetti

Il passaggio dalla **classe** (concetto astratto) all'**oggetto** (entità concreta) avviene tramite una **funzione speciale della classe:**

- detta **metodo costruttore**.

In un programma orientato agli oggetti:

- il codice **manipola oggetti**
- tutte le operazioni avvengono sugli oggetti.

Le classi come “fabbriche di oggetti”

Si può immaginare una classe come una **fabbrica di oggetti**, che:

- **definisce l’interfaccia degli oggetti verso il mondo esterno**
- **nasconde l’implementazione interna, incapsulando dati e funzionalità degli oggetti che produce**

Dal diagramma UML al codice

- L’**analista** identifica gli oggetti e produce il **diagramma delle classi UML**.
- Il **programmatore** prende le classi definite dall’analista e, tramite il **costruttore**, **istanzia gli oggetti** corrispondenti.

Il **metodo costruttore**:

- crea l’oggetto in memoria
- lo **inizializza** con i valori iniziali
  Esempio in Java

Java

```java
public class Studente {
    // Attributi e metodi della classe
}
```

- Questa istruzione dichiara una **classe pubblica** `Studente`.
- La classe include **implicitamente un costruttore di default se non ne viene dichiarato uno esplicitamente**.
- **La classe definisce sia come creare gli oggetti (costruttore) sia i comportamenti che gli oggetti possono eseguire (metodi della classe)**.

Metodi

- **I metodi sono funzioni associate a una classe.**
- Si chiamano metodi perché **appartengono agli oggetti della classe**.

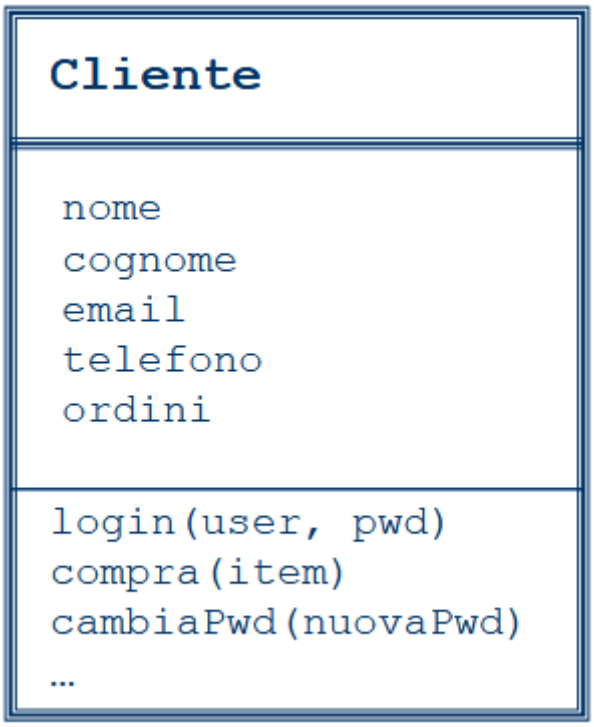
- Esempio:
  - **un oggetto `Cliente` può avere un metodo `cambiaPassword()` che agisce solo sull'oggetto specifico, non sull'intera classe.**

La classe `Cliente`

Riprendendo l'esempio di Amazon, uno dei **protagonisti del dominio** è il **Cliente**.

Se modelliamo il sistema con il paradigma **orientato agli oggetti (OOP)**:

- **la classe `Cliente` rappresenta il template di tutti i clienti**
- **definisce gli attributi e i metodi che ciascun oggetto cliente possiede**



In un diagramma UML:

1. **Nome della classe**
   - **Sempre al singolare**
   - **Indica l'entità astratta rappresentata**
2. **Attributi**
   - **Rappresentano le proprietà o caratteristiche di ogni oggetto cliente**
   - Ad esempio: nome, cognome, email, indirizzo, ecc.
3. **Metodi**
   - **Rappresentano le funzioni o comportamenti applicabili a ciascun oggetto cliente**

- Ad esempio: `acquistaProdotto()`, `aggiornaProfilo()`, `visualizzaOrdini()`
  Ogni **oggetto** `c:Cliente`:

- **è un'istanza della classe** `Cliente`
- **possiede i valori specifici degli attributi**
- **può eseguire i metodi definiti nella classe**

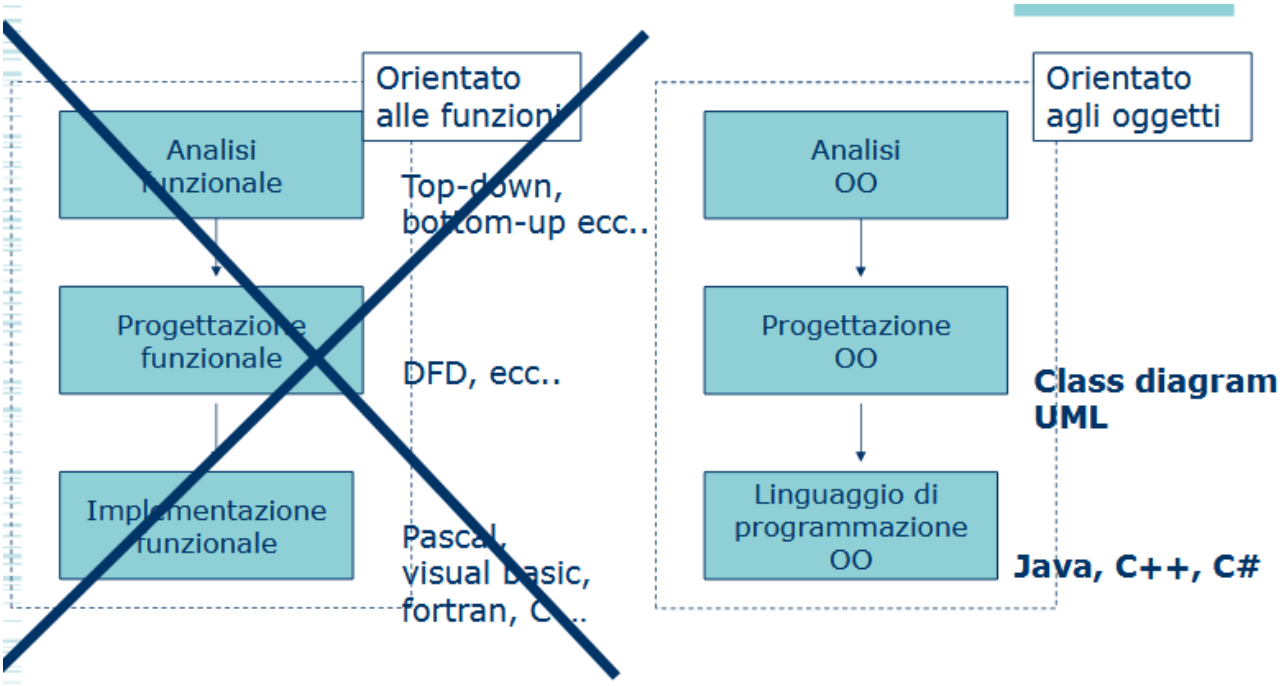
In questo modo, la **classe definisce cosa è possibile fare** con un cliente, mentre gli **oggetti rappresentano clienti concreti** all'interno del sistema.

Dal concetto astratto all'oggetto concreto

Finora abbiamo visto come la **classe** `Cliente` rappresenti un **modello astratto**:

- **definisce attributi e metodi comuni a tutti i clienti, senza rappresentare un cliente reale.**

Il passo successivo è **creare oggetti concreti**, cioè **istanze della classe**, che rappresentano clienti reali all'interno del sistema.



Come si passa dalla classe all'oggetto

1. **Classe**
   - **Definisce cosa è un cliente: attributi e comportamenti**
   - **È un concetto astratto (template)**
2. **Oggetto / Istanza**

- **È un cliente concreto, con valori specifici per ogni attributo**
- **Esegue i metodi della classe, agendo sui propri dati**

Sintesi

- La **classe** è come un **progetto o uno stampo**
- L'**oggetto** è il **prodotto reale** creato seguendo quel progetto
- In Java, la creazione di un oggetto avviene tramite il **costruttore** della classe

Esempio pratico (continuando con `Cliente`):

Java

```java
// Creazione di un oggetto Cliente
Cliente cliente1 = new Cliente("Marco", "Rossi", "marco@example.com");
// L'oggetto cliente1 ora ha attributi concreti
// e può usare i metodi definiti nella classe
cliente1.acquistaProdotto("Laptop");

```

Natura degli oggetti

Un **oggetto** è un'istanza concreta di una **classe**.

Caratteristiche principali

1. **Struttura analoga**
   - **Tutti gli oggetti di una stessa classe hanno la stessa struttura: gli stessi attributi.**
2. **Comportamento analogo**
   - **Tutti gli oggetti di una stessa classe possono eseguire gli stessi metodi, definiti nella classe.**
   - In Java, **l'oggetto non contiene fisicamente i metodi, ma solo i dati (gli attributi)**.
3. **Creazione e distruzione**
   - **La creazione dell'oggetto è sempre a carico del programmatore, tramite il costruttore della classe.**

- **La distruzione è spesso automatica, gestita dal garbage collector di Java.**

4. **Inizializzazione degli attributi**
   - Una volta istanziato, un oggetto **non può avere attributi "non definiti"**:
     - **tutti gli attributi assumono un valore di default se non specificato esplicitamente.**

- **Gli oggetti rappresentano entità concrete con valori specifici per gli attributi.**
- **I metodi rimangono definiti nella classe e sono condivisi da tutte le istanze**.

Stato dell'oggetto

Ogni **oggetto** possiede un **proprio stato**, rappresentato dai **valori dei suoi attributi**.

Caratteristiche principali

1. **Dati indipendenti**
   - **Ogni oggetto memorizza dati indipendenti dagli altri oggetti della stessa classe.**
2. **Variazione dello stato**
   - **Durante la vita dell'oggetto, lo stato può cambiare, perché i valori degli attributi possono essere modificati.**
   - **Le modalità con cui lo stato può variare sono stabilite dalla classe tramite i metodi disponibili.**
3. **Distinzione tra oggetti**
   - Due oggetti della stessa classe possono avere **gli stessi valori negli attributi** (stati identici),

    - **ma rimangono oggetti distinti in memoria.**

> **Sintesi**
>
> - Lo **stato** di un oggetto **è ciò che lo distingue concretamente dagli altri oggetti della stessa classe.**
> - I **metodi della classe** **definiscono come lo stato può cambiare, permettendo di manipolare gli attributi senza violare l'**incapsulamento.

Inventare i tipi

In **programmazione orientata agli oggetti (OO)** è possibile **definire i propri tipi di dati**, specificandone:

- **la struttura (attributi)**
- **i comportamenti (metodi) a corredo**

I nuovi tipi così creati possono essere **riutilizzati in altri programmi**, aumentando la modularità e la manutenibilità del codice.

Proprio come nella realtà, ogni oggetto ha:

- **un'identità**
- **dei comportamenti**
- **uno stato**

Le reference

In **programmazione orientata agli oggetti**, per **accedere agli oggetti** e manipolarli si utilizzano le **reference**.

- Una reference **è un riferimento o un puntatore a basso livello verso l'oggetto in memoria**.
- Possiamo immaginarla come un **"telecomando"**:
    - **tramite la reference possiamo comandare l'oggetto senza doverlo copiare o ricreare.**

Esempio

La classe `Date` della libreria Java rappresenta una data intesa come **istante di tempo** (data e ora).

```java
Date dataNascita;
```

- `dataNascita` è una **reference** di tipo `Date`
- **Non è ancora un oggetto concreto, ma può riferirsi a un oggetto Date in memoria**
- Per creare l'oggetto effettivo si usa il costruttore:

```java
Date dataNascita = new Date(2000, 1, 15);
```

Ora `dataNascita` **punta a un oggetto concreto**, e possiamo usare i suoi metodi.

Procedurale vs Object-Oriented

Paradigma	Descrizione	Relazione con i dati
Procedurale	Le funzioni sono moduli riusabili che operano sui dati	Le funzioni hanno priorità sui dati; i dati sono accessibili da tutte le funzioni
Object-Oriented	Le classi sono moduli riusabili che incapsulano dati e funzioni	I dati hanno priorità sulle funzioni; le funzioni agiscono solo sui dati per cui è previsto l'accesso tramite reference

Chiamata al costruttore

Per creare un oggetto in Java è necessario utilizzare:

1. **L'operatore** `new`
2. **Il metodo costruttore** della classe
   Esempio pratico:

```java
Date d = new Date();
```

- **Questa istruzione crea un'istanza della classe `Date` con la data e ora di sistema**.
- **La reference `d` punta ora all'oggetto appena creato**.
- Attraverso `d` è possibile **chiamare i metodi dell'oggetto**.

Manipolazione dei dati dell'oggetto

```java
d.setMonth(3); // Imposta il mese ad Aprile (i mesi sono indicizzati da 0)
int giorno = d.getDay(); // Restituisce il giorno della settimana come numero (0 = domenica, 6 = sabato)
```

- `setMonth(3)` **modifica lo stato dell'oggetto (mese)**.
- `getDay()` **restituisce un valore basato sullo stato corrente dell'oggetto**.

Sintesi

- La **reference** (`d`) permette di accedere all'oggetto e ai suoi metodi.
- **Il costruttore è il meccanismo che crea l'oggetto in memoria e lo inizializza.**
- **Dopo la creazione, l'oggetto può essere manipolato solo tramite la reference che lo rappresenta**.

Regole dei costruttori

Ogni classe in Java ha sempre **almeno un costruttore**, perché **è il meccanismo che permette di trasformare una classe astratta in un oggetto concreto.**

Spesso una classe, in Java, può prevedere **più costruttori:**

- **ognuno con parametri diversi, così da poter creare oggetti in modi differenti a seconda delle esigenze.**

I costruttori sono **funzioni speciali** che si riconoscono subito:

- **hanno sempre lo stesso nome della classe**
- **non dichiarano alcun tipo di ritorno**

Il loro scopo principale è **inizializzare l'oggetto:**

- **ovvero assegnare valori agli attributi in modo che l'oggetto nasca già pronto all'uso.**

Se non dichiariamo alcun costruttore, **Java crea automaticamente un costruttore di default, che inizializza gli attributi ai valori predefiniti**.
Ma se nel codice dichiariamo anche solo un costruttore personalizzato, quello di default **non viene più generato**, perché il compilatore assume che abbiamo già definito il modo corretto per inizializzare l'oggetto.

Lavorare con le reference

Come abbiamo anticipato sopra le **reference:**

- **permettono di accedere e manipolare gli oggetti in Java**,
- e hanno alcune proprietà importanti da ricordare.

1. **Reference null**
   - Una reference può essere impostata a `null` per indicare che **non fa ancora riferimento a nessun oggetto**.

Java

```java
Date d = null;
```

2. **Più reference sullo stesso oggetto**

- È possibile avere **due o più variabili che puntano allo stesso oggetto**.

Java

```java
Date data1 = new Date();
Date data2;
data2 = data1;
```

- **In questo caso `data2` non crea un nuovo oggetto, ma punta allo stesso oggetto di `data1`**.

3. **Cosa succede assegnando una reference a un'altra?**

- **Non si copia l'oggetto, si copia la reference**.
- **Quindi entrambe le variabili fanno riferimento allo stesso oggetto in memoria**.
- **Modificare l'oggetto tramite una reference influenzerà anche l'altra, perché puntano allo stesso oggetto**.

Esempio: reference e assegnazione

Esempio 1

Creiamo **due variabili di tipo** `Date` e le inizializziamo con **due oggetti distinti**:

Java

```java
// creo il 13 dicembre 2001
Date dataInverno = new Date(101, 11, 13);

// creo il 31 luglio 2005
Date dataEstate = new Date(105, 6, 31);
```

In questo caso:

- `dataInverno` → **fa riferimento a un oggetto Date che rappresenta il** `13 dicembre 2001`
- `dataEstate` → **fa riferimento a un altro oggetto Date che rappresenta il** `31 luglio 2005`

I due oggetti sono **distinti** e occupano **due aree diverse di memoria**, anche se appartengono alla stessa classe.

> ⚠ **Nota sul costruttore di** `Date`**:**
>
> - l'anno è espresso come `anno - 1900`
> - il mese è espresso con indice che parte da `0`
> - il giorno indica il giorno del mese

Esempio 2

Ora eseguiamo la seguente assegnazione:

Java

```
dataInverno = dataEstate;
```

Cosa succede?

- **Non viene copiato l'oggetto**
- **Viene copiata la reference**
- **Sia `dataInverno` sia `dataEstate` puntano allo stesso oggetto**

Di conseguenza:

- L'oggetto creato inizialmente per `dataInverno` **non è più referenziato da nessuna variabile**
- Questo oggetto diventa **candidato alla rimozione da parte del Garbage Collector**

> **Osservazione importante**
>
> Dopo l'assegnazione:
>
> - **esiste un solo oggetto `Date`**
> - **esistono due reference che lo indicano**
>
> Qualsiasi modifica fatta all'oggetto tramite `dataInverno` o `dataEstate` avrà effetto **sullo stesso oggetto in memoria**.

Il Garbage Collector (GC)

In Java, come in Python (anche se con differenze sostanziali nell'implementazione), la gestione della memoria è **automatica** ed è affidata alla JVM tramite il **Garbage Collector (GC)**.

Prima di tutto è importante distinguere **dove** vengono allocati i vari elementi:

- **I tipi primitivi (`int`, `double`, `boolean`, ecc.) vengono allocati direttamente nello stack**.
- Gli **oggetti**, invece, **vengono creati dinamicamente tramite l'operatore `new` e sono memorizzati nel Java Heap**.
- **Le reference agli oggetti si trovano anch'esse nello stack, non nel heap**.

Il Java Heap

- **è la porzione di RAM che contiene tutti gli oggetti creati dal programma.**

Ruolo del Garbage Collector

Il **Garbage Collector** è una routine di sistema che ha il compito di:

- **scandire periodicamente il Java Heap**
- **individuare gli oggetti non più referenziati**
- **liberare la memoria da essi occupata**

Un oggetto diventa “spazzatura” quando **nessuna reference lo punta più**.

Il GC:

- **opera in un thread indipendente**
- **ha bassa priorità**
- è un **daemon thread:**
  - **lavora in background senza interferire direttamente con il flusso principale del programma**

Controllo della memoria

In Java:

- **il programmatore non può deallocare esplicitamente un oggetto**
- **non esiste l’equivalente del `free()` di altri linguaggi**

È possibile invocare:

Java

```java
System.gc();
```

ma:

- **questa chiamata non garantisce l’esecuzione immediata del Garbage Collector**
- **serve solo a suggerire alla JVM di avviare il GC**
- **la decisione finale resta sotto il controllo della JVM**

Idea chiave

Questo comportamento **non è casuale**: i progettisti di Java (Sun/Oracle) hanno scelto esplicitamente che il programmatore Java:

- **crea gli oggetti**
- **gestisce le reference**
- **non si occupa direttamente della memoria**

La **liberazione della memoria** è quindi una responsabilità del **runtime** (JVM) e non del **codice applicativo**.

Passaggio dei parametri

In Java, quando si invocano metodi passando dei parametri, è importante distinguere **tra oggetti e tipi primitivi**, perché il comportamento è diverso.

- **Gli oggetti vengono passati tramite reference**
  - **Al metodo non arriva l'oggetto in sé, ma una copia della reference che punta a quell'oggetto**.
- Di conseguenza, **il metodo e il chiamante fanno riferimento allo stesso oggetto in memoria**.
  - **Se il metodo modifica lo stato dell'oggetto (cioè i valori dei suoi attributi), le modifiche sono visibili anche all'esterno, dopo il ritorno dal metodo**.
- Questo non significa che Java passi gli oggetti "per riferimento" in senso stretto, ma che:
  - **la** reference **è passata per valore**
  - **il valore copiato è l'indirizzo dell'oggetto**
- **I tipi primitivi**, invece, sono passati **per valore puro**.
  - **Il metodo riceve una copia del valore (** `int` **,** `double` **,** `boolean` **, ecc.) e ogni modifica resta locale al metodo, senza effetti sulla variabile originale.**

Implementare una classe

Una classe in Java definisce un **nuovo tipo di dato** e rappresenta il modello (astratto) degli oggetti che verranno creati a runtime.

La forma generale di una classe è la seguente:

```java
public class NomeClasse {
    // dichiarazione degli attributi
    // definizione dei metodi
}
```

All'interno della classe troviamo:

- **gli attributi**, **che descrivono lo stato degli oggetti**
- **i metodi**, **che descrivono il comportamento degli oggetti**

È possibile scrivere **più classi nello stesso file**, ma con una regola fondamentale:

- **solo una classe può essere** `public`
- **il nome del file deve coincidere con il nome della classe pubblica**

Le altre classi definite nello stesso file:

- **non sono public**
- **sono visibili solo all'interno dello stesso** package

Questo meccanismo permette di organizzare il codice mantenendo visibilità e incapsulamento sotto controllo.

Definizione di pacchetto

In Java, un **pacchetto (package):**

- **è una struttura che serve a organizzare le classi di una libreria o di un'applicazione.**

Dal punto di vista pratico, un pacchetto:

- **è una directory**
- **contiene i file** `.class` **generati dalla compilazione**
- **aiuta a mantenere il progetto ordinato e modulare**

Il concetto di package ha **due significati complementari**.

1. Organizzazione logica

- **Ogni package rappresenta uno spazio di nomi (namespace) distinto.**
- **Classi con lo stesso nome possono esistere in package diversi senza entrare in conflitto.**

2. Organizzazione fisica

- **A livello di file system, ogni package corrisponde a una cartella distinta sul disco.**
- **La struttura delle directory rispecchia la gerarchia dei package.**

Nome di un pacchetto

Il package a cui appartiene una classe viene dichiarato **all'interno del file** `.java` tramite l'istruzione:

Java

```java
package myPackage;
```

Questa istruzione:

- **deve essere la prima riga del file**
- **associa la classe a quel preciso package**
- **determina anche il percorso fisico in cui verrà salvato il file** `.class`

Il nome del package dovrebbe essere **il più possibile univoco**, per evitare collisioni con altre librerie.
Per questo motivo si usa spesso una **notazione gerarchica** basata sul dominio inverso, ad esempio:

```
it.azienda.progetto.modulo
```

In questo modo i package contribuiscono non solo all'organizzazione del codice, ma anche alla sua **riusabilità e manutenibilità** nel tempo.

Uso dei pacchetti

Per utilizzare una classe definita in un altro package è necessario **importarla** esplicitamente tramite l'istruzione `import`.

La sintassi generale è:

```java
import myPackage.MyClass;
```

Ad esempio, per usare la classe `Date` della libreria standard di Java:

```java
import java.util.Date;
```

Dopo l' `import`, la classe può essere utilizzata direttamente nel codice senza doverne specificare ogni volta il nome completo (fully qualified name).

Regole sui package in Java

Java impone alcune regole precise sull'uso dei package:

- **tutte le classi devono appartenere a un package**
- **se il programmatore non specifica esplicitamente un package, la classe viene assegnata automaticamente a un package implicito (default package)**

> **L'uso del package implicito è consentito, ma sconsigliato nei progetti reali, perché limita la riusabilità e la gestione delle dipendenze.**

Ruolo degli IDE di sviluppo

Gli ambienti di sviluppo (come Eclipse, IntelliJ, VS Code):

- **creano automaticamente la struttura di directory dei package in fase di compilazione**
- **salvano i file `.class` nelle cartelle corrispondenti al package dichiarato**

In questo modo lo sviluppatore lavora a livello logico (package e classi), mentre l'IDE e il compilatore si occupano della corretta organizzazione fisica dei file

Attributi di una classe

Gli **attributi** rappresentano:

- **lo stato degli oggetti di una classe e vengono dichiarati all'interno della classe stessa.**

La loro dichiarazione ha la forma:

```java
tipo1 nomeVariabile1;
tipo2 nomeVariabile2;
```

Il tipo di un attributo può essere:

- **un tipo primitivo (`int`, `double`, `boolean`, ...)**
- **una classe di libreria (ad esempio `Date`)**
- **una classe definita dal programmatore**

Per poter usare una classe che appartiene a un altro package è necessario **importarla**, a meno che non si trovi nel package di default o nello stesso package della classe corrente.

```java
import package1.Class1;   // importa una singola classe
import package2.*;        // importa tutte le classi del package
```

Gli attributi possono essere **accessi tramite la notazione punto**:

```java
nomeOggetto.attributo;
```

Dal punto di vista sintattico questa operazione è lecita, ma **è fortemente sconsigliata** in una buona progettazione OOP, perché:

- **espone direttamente lo stato interno dell'oggetto**
- **riduce l'**incapsulamento
- **rende la classe meno robusta e più difficile da mantenere**

Per questo motivo, in genere, gli attributi vengono resi non accessibili direttamente e manipolati tramite metodi(getter e setter).

Metodi di una classe

I **metodi** definiscono:

- **il comportamento degli oggetti di una classe.**

Un metodo:

- **può ricevere zero o più parametri di input**
- **può restituire un valore oppure non restituire nulla**

Nel caso in cui restituisca un valore, deve essere dichiarato il tipo di ritorno.
Se invece non restituisce alcun valore, si utilizza il tipo `void`.

La forma generale di un metodo è:

```java
public/private tipoRitornato nomeMetodo(elencoParametri) {
    // dichiarazioni di variabili locali
    // istruzioni
    // ...
    return valore; // opzionale, se il tipo non è void
}
```

I metodi possono essere richiamati su un oggetto tramite la **notazione punto**:

```java
nomeOggetto.metodo(parametri);
```

In questo modo il metodo opera **sullo stato dell'oggetto** a cui appartiene, rispettando il modello a oggetti introdotto dalla classe.

Il tipo `enum`

In Java, come in altri linguaggi (es: Python), il tipo `enum` **(enumerativo)** permette di definire una **categoria chiusa di valori:**

- **cioè un insieme predefinito e limitato di oggetti possibili.**
  Un `enum`:
- **specifica un nuovo tipo di dato**
- **consente di istanziare solo i valori dichiarati**
- **è particolarmente utile quando un attributo può assumere un numero finito di stati noti**

Gli oggetti definiti in un `enum` sono, per default:

- `static`
- `final`

Per questo motivo vengono chiamati **costanti enum**.

Il tipo `enum` è **typesafe**, cioè:

- **il compilatore garantisce che il valore assegnato appartenga al range definito**
- **non è possibile assegnare valori arbitrari o non previsti**

La definizione di un `enum` elenca **tutti i valori ammessi**:

Java

```java
enum GiornoSettimana {
    lunedi, martedi, mercoledi,
    giovedi, venerdi, sabato, domenica
}
```

L'accesso a uno dei valori dell'enum avviene tramite la notazione:

Java

```java
NomeEnum.valore
```

Esempio di utilizzo

Supponiamo di voler modellare un evento che si ripete durante la settimana. Uno degli attributi dell'evento sarà il **giorno della settimana**, rappresentato tramite un `enum`.

Java

```java
public class EventoSettimanale {
    private String nome;
    private String location;
    private GiornoSettimana giornoEvento;

    public EventoSettimanale(String nome, String location,
                             GiornoSettimana giornoEvento) {
        this.nome = nome;
        this.location = location;
        this.giornoEvento = giornoEvento;
```

```
    }
}
```

Nel `main` è possibile utilizzare l' `enum` in modo sicuro:

Java

```java
public class Main {
    public static void main(String[] args) {
        GiornoSettimana giorno = GiornoSettimana.sabato;

        EventoSettimanale evento =
            new EventoSettimanale("gruppo lettura", "Roma", giorno);
    }
}
```

In questo modo:

- il giorno dell'evento può assumere **solo valori validi**
- si evita l'uso di stringhe o numeri "magici"
- il codice risulta più **leggibile, robusto e manutenibile**

Metodi Statici degli `enum`

Ogni tipo `enum` in Java mette a disposizione **alcuni metodi di utilità**, forniti automaticamente dal linguaggio.
Si tratta di **metodi statici:**

- **non legati, quindi, a una singola istanza, ma alla classe `enum` stessa** (in Python questi metodi li chiameremo metodi di classe).

I due metodi principali sono i seguenti:

1. `valueOf(String nome)`

Java

```java
NomeEnum valueOf(String nome)
```

Questo metodo:

- riceve una `String`

- **restituisce la costante enum il cui nome coincide con la stringa passata**

Esempio:

```java
GiornoSettimana g = GiornoSettimana.valueOf("sabato");
```

Se la stringa **non corrisponde esattamente** a una delle costanti definite nell'enum, viene sollevata un'eccezione a runtime (`IllegalArgumentException`).

2. `values()`

```java
NomeEnum[] values()
```

Questo metodo:

- **restituisce un array contenente tutte le costanti dell'enum**
- **l'ordine dell'array è lo stesso in cui i valori sono stati dichiarati**

Esempio:

```java
GiornoSettimana[] giorni = GiornoSettimana.values();

for (GiornoSettimana g : giorni) {
    System.out.println(g);
}
```

Questo metodo è molto utile per:

- **iterare su tutti i valori possibili**
- **costruire menu, controlli o validazioni**
- **gestire logiche basate su un insieme finito di stati**

Metodi di istanza degli `enum`

Oltre ai metodi statici, ogni `enum` mette a disposizione anche **metodi di istanza:**

- **cioè metodi che possono essere invocati direttamente su una singola costante enum.**

Dato, ad esempio:

```java
GiornoSettimana g = GiornoSettimana.sabato;
```

è possibile utilizzare i seguenti metodi:

1. `name()`

```java
String name()
```

Restituisce una `String` contenente il nome esatto della costante enum, così come è stata dichiarata.

```java
g.name();   // "sabato"
```

2. `ordinal ()`

```java
int ordinal()
```

Restituisce la posizione della costante all'interno della dichiarazione dell'enum.
La numerazione parte da 0.

```java
g.ordinal();   // dipende dalla posizione di "sabato" nella enum
```

Questo valore è legato all'ordine di dichiarazione e **non dovrebbe essere usato come identificatore logico**, perché può cambiare se si modifica l'enum.

3. `compareTo(NomeEnum altro)`

```java
int compareTo(NomeEnum altro)
```

Confronta due costanti enum in base al loro ordine di dichiarazione.

- **restituisce un valore negativo se la costante chiamante precede quella passata come parametro**
- **restituisce 0 se sono la stessa costante**
- **restituisce un valore positivo se la costante chiamante segue quella parametro**

Java

```java
GiornoSettimana.lunedi.compareTo(GiornoSettimana.venerdi);
```

4. `toString()`

Java

```java
String toString()
```

Restituisce una rappresentazione testuale della costante enum.
Di default coincide con `name()`, ma **è l'unico metodo che può essere sovrascritto (override).**

Questo permette di:

- **fornire una stampa più leggibile**
- **separare il nome interno dell'enum dalla sua rappresentazione esterna**

Enum avanzate

In Java le `enum` non si limitano a definire un insieme di costanti:

- **è possibile associare dati e comportamenti a ciascuna costante, rendendole veri e propri oggetti.**

Per farlo, **si specificano dei valori tra parentesi accanto a ogni costante dell'enum.**

Esempio:

Java

```java
enum Taglia {
    small(42), medium(44), large(46);
```

```
}
```

In questo caso, a ogni valore dell'enum viene associata una **misura numerica**.

Attributi e costruttore dell'enum

Per memorizzare il valore associato alle costanti è necessario:

1. **Dichiarare un attributo**

```java
private int misura;
```

2. **Definire un costruttore**

```java
private Taglia(int misura) {
    this.misura = misura;
}
```

I costruttore viene invocato automaticamente **una sola volta per ogni costante enum**, al momento del caricamento della classe.

Nota importante:

- **il costruttore di un `enum` è sempre `private` (implicitamente o esplicitamente)**
- **non è possibile istanziare manualmente un enum con `new`**

Metodi dell'enum

È possibile aggiungere **metodi**, ad esempio getter (e, tecnicamente, anche setter):

```java
public int getMisura() {
    return misura;
}
```

Gli attributi associati alle costanti enum possono quindi:

- **essere letti**
- **ed eventualmente modificati, anche se in genere è buona pratica mantenerli immutabili**

Osservazioni

- Un `enum` è, di default, un tipo **speciale e controllato**
- Ogni costante è un **oggetto unico**
- Gli enum combinano:
  - sicurezza di tipo
  - valori finiti
  - dati e comportamenti associati

Per questo motivo sono molto usati per rappresentare **stati, categorie, configurazioni e opzioni** in modo robusto e leggibile.